



Case Study: Building a Local RAG (Retriever-Augmented Generation) System

Case Study Information

A new client working in the legal sector has requested a question-answering system that can help users find accurate answers from a collection of Arabic legal documents. Due to privacy concerns, the entire solution must run locally, with no dependence on cloud-based services or internet access.

Objective

The goal of this task is to build a complete Retriever-Augmented Generation (RAG) pipeline that takes raw documents, processes them, and enables users to ask questions and receive generated responses using a locally hosted language model.

Your Tasks

Work through the following components step by step to complete the system:

1. Document Preparation and Chunking

- Load documents from the supported formats (PDF, DOCX, TXT).
 - Implement two chunking methods:
 - Splitter-based chunking (using delimiters like newlines, punctuation, or custom rules).
 - Token-based chunking (e.g., splitting the text every 400 tokens).
 - Compare both approaches and summarize the differences in how they affect retrieval quality.
-

2. Embedding and Indexing

- Use an Arabic-compatible embedding model (such as those from `sentence-transformers`) to convert chunks into vectors.
 - Store these embeddings in a local vector store such as **FAISS** or **ChromaDB** for fast retrieval.
-



3. LLM Integration (Ollama)

- Install and run a local LLM using **Ollama**.
 - Select an available model that supports Arabic (e.g., `gemma`, `mistral`).
 - Format prompts by combining the retrieved chunks with the user question before passing them to the model.
 - Handle and display the response clearly.
-

4. Retrieval-Augmented QA Pipeline

- Set up a retrieval system that:
 - Takes the user's question.
 - Retrieves the most relevant chunks from the vector store.
 - Combines the question and context into a prompt.
 - Sends it to the local LLM and returns the response.
-

5. Evaluation and Testing

- Prepare several Arabic questions to test the system.
 - Test examples where the answers are accurate, vague, or incorrect.
 - Rate and document the system's performance.
-

Deliverables

- Fully working RAG system (script or notebook format).
- Presentation summarizing:
 - System architecture and components.
 - Visual diagrams.
 - Test results and analysis.
- (Optional) Build a simple front-end interface (Streamlit or Gradio) to make testing easier.